Intrinsically Recursive Coalgebras

Cass Alexandru[1] Henning Urbat[2] Thorsten Wißmann[2]

[1]RPTU Kaiserslautern-Landau & Radboud University Nijmegen

[2]FAU Erlangen-Nürnberg

TYPES 2026

Motivation

Motivation

- Divide-and-Conquer algorithms are captured by the category-theoretical notion of *coalgebra-to-algebra morphisms*

Motivation

- Divide-and-Conquer algorithms are captured by the category-theoretical notion of *coalgebra-to-algebra morphisms*
- In the context of *total functional programming* (i.e. algorithms proven to terminate), these arise from *recursive coalgebras*

Motivation

- Divide-and-Conquer algorithms are captured by the category-theoretical notion of *coalgebra-to-algebra morphisms*
- In the context of *total functional programming* (i.e. algorithms proven to terminate), these arise from *recursive coalgebras*
- However: Lack of criteria for proving "recursivity" of a coalgebra, in particular amenable to formalization in a dependetly typed fp language (s.a. Agda)

Motivation

- Divide-and-Conquer algorithms are captured by the category-theoretical notion of *coalgebra-to-algebra morphisms*
- In the context of *total functional programming* (i.e. algorithms proven to terminate), these arise from *recursive coalgebras*
- However: Lack of criteria for proving “recursivity” of a coalgebra, in particular amenable to formalization in a dependetly typed fp language (s.a. Agda)
- This talk:
  - Motivation for Divide-and-Conquer Algorithms, categorically

Motivation

- Divide-and-Conquer algorithms are captured by the category-theoretical notion of *coalgebra-to-algebra morphisms*
- In the context of *total functional programming* (i.e. algorithms proven to terminate), these arise from *recursive coalgebras*
- However: Lack of criteria for proving "recursivity" of a coalgebra, in particular amenable to formalization in a dependetly typed fp language (s.a. Agda)
- This talk:
  - Motivation for Divide-and-Conquer Algorithms, categorically
  - How proving partial correctness sets the stage for expressing...

Motivation

- Divide-and-Conquer algorithms are captured by the category-theoretical notion of *coalgebra-to-algebra morphisms*
- In the context of *total functional programming* (i.e. algorithms proven to terminate), these arise from *recursive coalgebras*
- However: Lack of criteria for proving "recursivity" of a coalgebra, in particular amenable to formalization in a dependetly typed fp language (s.a. Agda)
- This talk:
  - Motivation for Divide-and-Conquer Algorithms, categorically
  - How proving partial correctness sets the stage for expressing...
  - our novel categorical criterion for termination of such algorithms!

Structure

Divide and Conquer "Divide and Conquer"

- A D&C algorithm can be split into the following steps:

Divide and Conquer “Divide and Conquer”

- A D&C algorithm can be split into the following steps:
  - *Divide* input into “smaller”[1] inputs;

[1] this is a Chekov’s gun

Divide and Conquer "Divide and Conquer"

- A D&C algorithm can be split into the following steps:
  - *Divide* input into "smaller"[1] inputs;
  - Recursively apply the algorithm to them;

[1]this is a Chekov's gun

Divide and Conquer “Divide and Conquer”

- A D&C algorithm can be split into the following steps:
  - *Divide* input into “smaller”[1] inputs;
  - Recursively apply the algorithm to them;
  - *Combine* to compute the result.

[1]this is a Chekov’s gun

Case Studies: Quick- and Mergesort

- Algorithms can differ in which step does the “heavy lifting”

Case Studies: Quick- and Mergesort

- Algorithms can differ in which step does the "heavy lifting"
- Quicksort: Main logic in the *divide* step: partition elements around the pivot. Combine step: concatenation.

Case Studies: Quick- and Mergesort

- Algorithms can differ in which step does the "heavy lifting"
- Quicksort: Main logic in the *divide* step: partition elements around the pivot. Combine step: concatenation.
- Mergesort: Business end is the *combine* step: zipping two ordered lists into one. Divide step: Splitting the list in half.

D&CAs as Coalgebra-to-Algebra Morphisms

$$\begin{tikzcd}
FI \arrow[r, "Fh"] & FO \arrow[d, "a"] \\
I \arrow[u, "c"] \arrow[r, dashed, "h"] & O
\end{tikzcd}$$

D&CAs as Coalgebra-to-Algebra Morphisms

$$\begin{tikzcd} FI \arrow[r, "Fh"] & FO \arrow[d, "a"] \\ I \arrow[u, "c"] \arrow[r, dashed, "h"] & O \end{tikzcd}$$

- *Coalgebra* c : Divide input up into smaller inputs, the distribution of which is given by a functor F;

D&CAs as Coalgebra-to-Algebra Morphisms

$$\begin{tikzcd}
FI \arrow[r, "Fh"] & FO \arrow[d, "a"] \\
I \arrow[u, "c"] \arrow[r, dashed, "h"] & O
\end{tikzcd}$$

- *Coalgebra* c : Divide input up into smaller inputs, the distribution of which is given by a functor F;
- Fh: Apply h recursively under F;

D&CAs as Coalgebra-to-Algebra Morphisms

$$\begin{tikzcd} FI \arrow[r, "Fh"] & FO \arrow[d, "a"] \\ I \arrow[u, "c"] \arrow[r, dashed, "h"] & O \end{tikzcd}$$

- *Coalgebra* c : Divide input up into smaller inputs, the distribution of which is given by a functor F;
- Fh: Apply h recursively under F;
- *Algebra* a: Combine an F-structure of the results of recursive calls to obtain the output.

D&CAs as Coalgebra-to-Algebra Morphisms

$$\begin{tikzcd} FI \arrow[r, "Fh"] & FO \arrow[d, "a"] \\ I \arrow[u, "c"] \arrow[r, dashed, "h"] & O \end{tikzcd}$$

- A coalgebra c is called *recursive* if, for every algebra a, it admits a unique solution to the equation[2]

$$h = c; Fh; a$$

[2] sometimes called the "hylo" or *hylomorphism* equation

D&CAs as Coalgebra-to-Algebra Morphisms

$$\begin{tikzcd} FI \arrow[r, "Fh"] & FO \arrow[d, "a"] \\ I \arrow[u, "c"] \arrow[r, dashed, "h"] & O \end{tikzcd}$$

- A coalgebra c is called *recursive* if, for every algebra a, it admits a unique solution to the equation[2]

$$h = c; Fh; a$$

- NB: In a language permitting general recursion, the above may be read as a *definition*.

[2] sometimes called the "hylo" or *hylomorphism* equation

Structure

Narrowing our focus

- As mentionend, the *interesting* step of quicksort is the *divide* step, i.e. partitioning a list around a pivot.

Narrowing our focus

- As mentionend, the *interesting* step of quicksort is the *divide* step, i.e. partitioning a list around a pivot.
- We therefore focus on that step from now.

Narrowing our focus

- As mentionend, the *interesting* step of quicksort is the *divide* step, i.e. partitioning a list around a pivot.
- We therefore focus on that step from now.
- partition: $\mathsf{List}A \to 1 + \mathsf{List}A \times A \times \mathsf{List}A \ldots$

Narrowing our focus

- As mentionend, the *interesting* step of quicksort is the *divide* step, i.e. partitioning a list around a pivot.
- We therefore focus on that step from now.
- $\mathsf{partition} : \mathsf{List} A \to 1 + \mathsf{List} A \times A \times \mathsf{List} A$ ...
- ... $\Rightarrow$ Functor F is: $FX = 1 + X \times A \times X$

Example: Growing a BST with partition

`2 : 5 : 4 : 1 : 3 : []`

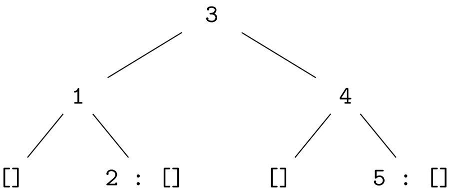
$\mathsf{partition} \colon \mathsf{List} A \to \bigcirc + \mathsf{List} A \times A \times \mathsf{List} A$

Example: Growing a BST with partition

```
          3
        /   \
2 : 1 : []   5 : 4 : []
```

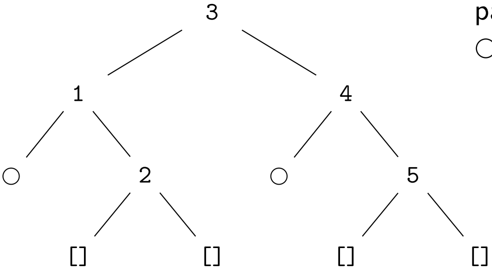
$$\mathsf{partition}\colon \mathsf{List}A \to \bigcirc + \mathsf{List}A \times A \times \mathsf{List}A$$

Example: Growing a BST with partition



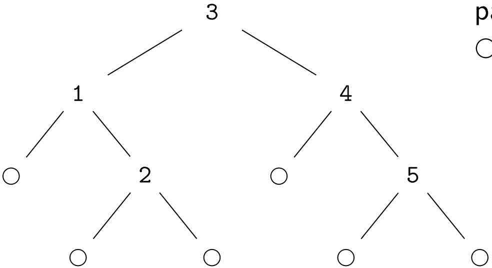
$\mathsf{partition} \colon \mathsf{List}A \to$
$\bigcirc + \mathsf{List}A \times A \times \mathsf{List}A$

Example: Growing a BST with partition



$\mathsf{partition} : \mathsf{List}A \to$
$\bigcirc + \mathsf{List}A \times A \times \mathsf{List}A$

Example: Growing a BST with partition



$\text{partition} : \text{List}A \to \bigcirc + \text{List}A \times A \times \text{List}A$

Partial Correctness of Quicksort

- Orderedness: The elements to the left/right of the pivot in the $\mathsf{List}\,A \times (p : A) \times \mathsf{List}\,A$ case are smaller/greater than p
- Element-preservation: $\mathsf{partition}(xs)$ and xs have the same multiset of elements.
- Working in the setting of data with mappings to the multiset ($\mathcal{B}A$) of their elements allows us to express both these properties!
- We focus on the element-preservation property here, as that is relevant for termination.

Sliced Partition

- Redefine partition in the slice category Set$/\mathcal{B}A$
- We can define lift F to $\bar{F}$ as:

$$\bar{F}\begin{pmatrix} X \\ f \end{pmatrix} := \begin{pmatrix} 1 \\ \emptyset \end{pmatrix} + \begin{pmatrix} \{(l,p,r) \in X \times A \times X\} \\ f(l) \uplus \{p\} \uplus f(r) \end{pmatrix}$$

- Note: The multiset indices of the recursive positions are smaller than the outer index: $|f(l)|, |f(r)| < |f(l) \uplus \{p\} \uplus f(r)|$.

Structure

Formalizing "the indices of the recursive positions are smaller than the outer index"

Notation

Fix a well order $(W, <)$. For $i \in W$, we denote by ${<}i := \{j \in W \mid j < i\}$ the set of indices strictly smaller than i (the *downset* of i). We have two projection functors, *restriction* and *evaluation*:

$$\begin{aligned} - \,|_{<i} &: \mathcal{C}^W \to \mathcal{C}^{<i} & \qquad \mathsf{ev}_i &: \mathcal{C}^W \to \mathcal{C} \\ X\,|_{<i} &:= (X_j)_{j \in {<}i} & \qquad \mathsf{ev}_i X &:= X_i \end{aligned}$$

Introducing: Well Founded Functors

- Intuition: "The ith output of the functor $F\colon \mathcal{C}^W \to \mathcal{C}^W$ is fully determined by its inputs with indices $j < i$."

Introducing: Well Founded Functors

- Intuition: "The ith output of the functor $F\colon \mathcal{C}^W \to \mathcal{C}^W$ is fully determined by its inputs with indices $j < i$."
- "$F\colon \mathcal{C}^{j\in W} \to \mathcal{C}^{i\in W}$ is morally equivalent to a family $(F_{<i}\colon \mathcal{C}^{j<i} \to \mathcal{C})_{i\in W}$"

Introducing: Well Founded Functors

Definition (Well-Founded Functor)

A functor $F\colon \mathcal{C}^W \to \mathcal{C}^W$ is *well-founded* if for every $i \in W$, the composition $\mathsf{ev}_i \cdot F\colon \mathcal{C}^W \to \mathcal{C}$ factors through the restriction $|_{<i}\colon \mathcal{C}^W \to \mathcal{C}^{<i}$, that is, there exists a functor $F_{<i}\colon$ such that the diagram below commutes up to natural isomorphism:

$$\forall i \in W\colon \quad \begin{tikzcd} \mathcal{C}^W \arrow[r, "F"] \arrow[d, "-|_{<i}"'] \arrow[dr, phantom, "\cong"] & \mathcal{C}^W \arrow[d, "\mathsf{ev}_i"] \\ \mathcal{C}^{<i} \arrow[r, dashed, "\exists F_{<i}"] & \mathcal{C} \end{tikzcd}$$

Proving WFness of a functor

$$\begin{tikzcd}
\mathcal{C}^W \arrow[r, "F"] \arrow[d, "-|_{<i}"'] \arrow[dr, phantom, "\cong"] & \mathcal{C}^W \arrow[d, "\mathsf{ev}_i"] \\
\mathcal{C}^{<i} \arrow[r, dashed, "F_{<i}"] & \mathcal{C}
\end{tikzcd}$$

Proving WFness of a functor

$$\begin{tikzcd} \mathcal{C}^W \arrow[r, "F"] \arrow[d, "-|_{<i}"'] \arrow[dr, phantom, "\cong"] & \mathcal{C}^W \arrow[d, "\mathsf{ev}_i"] \\ \mathcal{C}^{<i} \arrow[r, dashed, "F_{<i}"] & \mathcal{C} \end{tikzcd}$$

- We define a canonical way to turn any functor F into a family $F_{<i} : (\mathcal{C}^{<i} \to \mathcal{C})_{i \in W}$, for which we obtain a projection $\varepsilon_F X i : F_{<i}(X|_{<i}) i \to F X i$.

Proving WFness of a functor

$$\begin{tikzcd} \mathcal{C}^W \arrow[r, "F"] \arrow[d, "-|_{<i}"'] \arrow[dr, phantom, "\cong"] & \mathcal{C}^W \arrow[d, "\mathsf{ev}_i"] \\ \mathcal{C}^{<i} \arrow[r, dashed, "F_{<i}"] & \mathcal{C} \end{tikzcd}$$

- We define a canonical way to turn any functor F into a family $F_{<i} \colon (\mathcal{C}^{<i} \to \mathcal{C})_{i \in W}$, for which we obtain a projection $\varepsilon_F X i : F_{<i}(X|_{<i}) i \to F X i$.
- Client code of the library then consists of definining an inclusion $\varepsilon_F^{-1} X i : F X i \to F\!< i(X|_{<i}) i$ which is an inverse to this.

Proving WFness of a functor

$$
\begin{tikzcd}
\mathcal{C}^W \arrow[r, "F"] \arrow[d, "-|_{<i}"'] \arrow[dr, phantom, "\cong"] & \mathcal{C}^W \arrow[d, "\mathsf{ev}_i"] \\
\mathcal{C}^{<i} \arrow[r, dashed, "F_{<i}"] & \mathcal{C}
\end{tikzcd}
$$

- We define a canonical way to turn any functor F into a family $F_{<i} \colon (\mathcal{C}^{<i} \to \mathcal{C})_{i \in W}$, for which we obtain a projection $\varepsilon_F X i : F_{<i}(X|_{<i})i \to FXi$.
- Client code of the library then consists of definining an inclusion $\varepsilon_F^{-1} X i : FXi \to F{<}\ i(X|_{<i})i$ which is an inverse to this.
- Next: How we use these parts to solve our original goal of proving recursivity for coalgebras for F

Diagrammatically

$$
\forall i \in W\colon \quad
\begin{tikzcd}
(F_{<i} C \,|_{<i})_i \arrow[r, "(F_{<i} h|_{<i})_i"] \arrow[dr, phantom, "\circlearrowright"] & (F_{<i} D \,|_{<i})_i \arrow[d, "\varepsilon_i"] \\
F C_i \arrow[u, red, "\varepsilon_i^{-1}"] \arrow[r, "F h_i"'] & F D_i \arrow[d, "a_i"] \\
C_i \arrow[u, "c_i"] \arrow[r, "h_i"] & D_i
\end{tikzcd}
$$

Diagrammatically

$$\forall i \in W\colon \quad \begin{tikzcd}
(F_{<i} C \mid_{<i})_i \arrow[r, "(F_{<i} h|_{<i})_i"] \arrow[dr, phantom, "\circlearrowright"] & (F_{<i} D \mid_{<i})_i \arrow[d, "\varepsilon_i"] \\
F C_i \arrow[u, red, "\varepsilon_i^{-1}"] \arrow[r, "F h_i"'] & F D_i \arrow[d, "a_i"] \\
C_i \arrow[u, "c_i"] \arrow[r, "h_i"] & D_i
\end{tikzcd}$$

- To define h_i, we need only $h \mid_{<i}$ ...

Diagrammatically

$$\forall i \in W\colon \quad \begin{tikzcd}
(F_{<i} C \,|_{<i})_i \arrow[r, "(F_{<i} h|_{<i})_i"] \arrow[dr, phantom, "\circlearrowleft"] & (F_{<i} D \,|_{<i})_i \arrow[d, "\varepsilon_i"] \\
FC_i \arrow[u, "\varepsilon_i^{-1}"] \arrow[r, "Fh_i"'] & FD_i \arrow[d, "a_i"] \\
C_i \arrow[u, "c_i"] \arrow[r, "h_i"] & D_i
\end{tikzcd}$$

- To define h_i, we need only $h \,|_{<i}$ ...
- We can define $(h_i)_{i \in W}$ by well founded induction!

Diagrammatically

$$\forall i \in W\colon \quad \begin{tikzcd}
(F_{<i} C\,|_{<i})_i \arrow[r, "(F_{<i} h|_{<i})_i"] \arrow[dr, phantom, "\circlearrowright"] & (F_{<i} D\,|_{<i})_i \arrow[d, "\varepsilon_i"] \\
F C_i \arrow[u, "\varepsilon_i^{-1}", red] \arrow[r, "F h_i"'] & F D_i \arrow[d, "a_i"] \\
C_i \arrow[u, "c_i"] \arrow[r, "h_i"] & D_i
\end{tikzcd}$$

- To define h_i, we need only $h\,|_{<i}$ ...
- We can define $(h_i)_{i \in W}$ by well founded induction!
- Next: How we define $F_{<i}$

Wellfoundification

$$\begin{tikzcd}
\mathcal{C}^W \arrow[r, "F"] \arrow[d, "-|_{<i}"] \arrow[dr, phantom, "\overset{!}{\cong}"] & \mathcal{C}^W \arrow[d, "\mathsf{ev}_i"] \\
\mathcal{C}^{<i} \arrow[r, dashed, "F_{<i}"] \arrow[d, "J_{<i}"] & \mathcal{C} \\
\mathcal{C}^W \arrow[r, "F"] & \mathcal{C}^W \arrow[u, "\mathsf{ev}_i"']
\end{tikzcd}$$

Definition (Inclusion Functor $J_<$)

$$(J_{<i}X)_j \colon \mathcal{C}^{<i} \to \mathcal{C}^W$$

$$(J_{<i}X)_j := \begin{cases} X_j & \text{if } j < i, \\ 0 & \text{otherwise} \end{cases}$$

Wellfoundification

$$
\begin{tikzcd}
\mathcal{C}^W \arrow[r, "F"] \arrow[d, "-|_{<i}"] & \mathcal{C}^W \arrow[d, "\mathsf{ev}_i"] \\
\mathcal{C}^{<i} \arrow[r, dashed, "\underset{F_{<i}}{\overset{!}{\cong}}"] \arrow[d, "J_{<i}"] & \mathcal{C} \\
\mathcal{C}^W \arrow[r, "F"] & \mathcal{C}^W \arrow[u, "\mathsf{ev}_i"']
\end{tikzcd}
$$

Definition (Inclusion Functor $J_<$)

$$(J_{<i}X)_j \colon \mathcal{C}^{<i} \to \mathcal{C}^W$$

$$(J_{<i}X)_j := \begin{cases} X_j & \text{if } j < i, \\ 0 & \text{otherwise} \end{cases}$$

- N.B.: Need decidability of $<?$ to define $J_{<i}$. Then, to prove inhabitation of $(J_{<i}X)_j$, one need prove $j <?\, i$ evaluate to $\top$.

Wellfoundification

$$
\begin{tikzcd}
\mathcal{C}^W \arrow[r, "F"] \arrow[d, "-|_{<i}"] & \mathcal{C}^W \arrow[d, "\mathrm{ev}_i"] \\
\mathcal{C}^{<i} \arrow[r, dashed, "!\cong" above, "F_{<i}" below] \arrow[d, "J_{<i}"] & \mathcal{C} \\
\mathcal{C}^W \arrow[r, "F"] & \mathcal{C}^W \arrow[u, "\mathrm{ev}_i"']
\end{tikzcd}
$$

Definition (Inclusion Functor $J_<$)

$$(J_{<i}X)_j \colon \mathcal{C}^{<i} \to \mathcal{C}^W$$

$$(J_{<i}X)_j := \begin{cases} X_j & \text{if } j < i, \\ 0 & \text{otherwise} \end{cases}$$

- N.B.: Need decidability of $<?$ to define $J_{<i}$. Then, to prove inhabitation of $(J_{<i}X)_j$, one need prove $j <? i$ evaluate to $\top$.
- We avoid this by specializing $\mathcal{C}$ to Set, and using *copartial elements*:

  $(J_{<i}X)_j \colon \mathsf{Set}^{<i} \to \mathsf{Set}^W \qquad (J_{<i}X)_j := \{x \mid j < i, x \in X_j\}$

Mechanization

$$\begin{tikzcd}
\mathcal{C}^W \arrow[r, "-|_{<i}"] \arrow[d, "F"] \arrow[dr, phantom, "\overset{!}{\cong}"] & \mathcal{C}^{<i} \arrow[r, "J_{<i}"] \arrow[d, dashed, "F_{<i}"] & \mathcal{C}^W \arrow[d, "F"] \\
\mathcal{C}^W \arrow[r, "\mathsf{ev}_i"'] & \mathcal{C} & \mathcal{C}^W \arrow[l, "\mathsf{ev}_i"]
\end{tikzcd}$$

```
< : A → Type -- downset
< i = Σ[ j ∈ A ] (j < i)
-- restriction
_|<_ : (A → Type) → (i : A) → ((< i) → Type)
(X |< i) (j , _pf) = X j

-- inclusion: copartial element formulation
J< : (i : A) → ((< i) → Type) → (A → Type)
J< i X j = Σ[ pf ∈ j < i ] X (j , pf)

-- truncation: restriction, then inclusion: _|<;J< ≈ T
T : (i : A) → (A → Type) → (A → Type)
T i X j = (j < i) × X j -- "annotate with pfs j < i"
```

Structure

Going Back to Definition-Time with Inversion

$$
\begin{array}{l}
\text{data } \mathsf{T}\ (X : \mathcal{B}\,A \to \mathsf{Type}) : \mathcal{B}\,A \to \mathsf{Type} \text{ where} \\
\quad \mathsf{leaf} : \mathsf{T}\ X\ [] \\
\quad \_|\lceil\_\rceil|\_ : \{i_l\ i_r : \mathcal{B}\,A\} \to (u : X\ i_l) \to (p : A) \to \\
\qquad (v : X\ i_r) \to \mathsf{T}\ X\ (p :: i_l +\!+\ i_r) \\
\text{pattern } \_\hat{}\_|\lceil\_\rceil|\_\hat{}\_\ u\ i_l\ p\ v\ i_r = \_|\lceil\_\rceil|\_\ \{i_l\}\ \{i_r\}\ u\ p\ v
\end{array}
$$

$$
\mathsf{T}\text{-}\varepsilon^{-1} : \{X : \mathcal{B}\,A \to \mathsf{Type}\} \to (i : \mathcal{B}\,A) \to \mathsf{T}\ X\ i \to \mathsf{T}\ (\mathsf{J}\!<\ i\ (X\ |\!<\ i))\ i
$$

$$
\begin{array}{l}
\mathsf{T}\text{-}\varepsilon^{-1}\ .[] \qquad \mathsf{leaf} \qquad = \mathsf{leaf} \\
\mathsf{T}\text{-}\varepsilon^{-1}\ .(p :: i_l +\!+\ i_r)\ (u\,\hat{}\,i_l\ |\lceil\ p\ \rceil|\ v\,\hat{}\,i_r) = \\
\quad ((\mathrm{i}_l\!<\!\mathrm{i}\ ,\ u)\ |\lceil\ p\ \rceil|\ (\mathrm{i}_r\!<\!\mathrm{i}\ ,\ v)) \text{ where } \mathrm{i}_l\!<\!\mathrm{i} : (i_l <_{\#} p :: i_l +\!+\ i_r)\ ;\ \mathrm{i}_r\!<\!\mathrm{i} : (i_r <_{\#} p :: i_l +\!+\ i_r)
\end{array}
$$

Going Back to Definition-Time with Inversion

$$\begin{array}{l}
\text{data } \mathsf{T}\ (X : \mathcal{B}\, A \to \mathsf{Type}) : \mathcal{B}\, A \to \mathsf{Type} \text{ where} \\
\quad \mathsf{leaf} : \mathsf{T}\ X\ [] \\
\quad _|\lceil_\rceil|_ : \{i_l\ i_r : \mathcal{B}\, A\} \to (u : X\ i_l) \to (p : A) \to \\
\qquad (v : X\ i_r) \to \mathsf{T}\ X\ (p :: i_l ++ i_r) \\
\text{pattern } _\frown_|\lceil_\rceil|_\frown_\ u\ i_l\ p\ v\ i_r = _|\lceil_\rceil|_\ \{i_l\}\ \{i_r\}\ u\ p\ v
\end{array}$$

$$\mathsf{T}\text{-}\varepsilon^{-1} : \{X : \mathcal{B}\, A \to \mathsf{Type}\} \to (i : \mathcal{B}\, A) \to \mathsf{T}\ X\ i \to \mathsf{T}\ (\mathsf{J}{<}\ i\ (X\ |{<}\ i))\ i$$

(1) Pattern match

$$\begin{array}{l}
\mathsf{T}\text{-}\varepsilon^{-1}\ .[]\ \mathsf{leaf} = \mathsf{leaf} \\
\mathsf{T}\text{-}\varepsilon^{-1}\ .(p :: i_l ++ i_r)\ (u \frown i_l\, |\lceil p \rceil|\, v \frown i_r) = \\
\quad ((\mathrm{i}_l{<}\mathrm{i}\ ,\ u)\ |\lceil p \rceil|\ (\mathrm{i}_r{<}\mathrm{i}\ ,\ v)) \text{ where } \mathrm{i}_l{<}\mathrm{i} : (i_l <_{\#} p :: i_l ++ i_r)\ ;\ \mathrm{i}_r{<}\mathrm{i} : (i_r <_{\#} p :: i_l ++ i_r)
\end{array}$$

1. Pattern match on the value of type $\mathsf{T}\ X\ i$;

Going Back to Definition-Time with Inversion

$$
\begin{array}{l}
\text{data } \mathsf{T}\ (X : \mathcal{B}\,A \to \mathsf{Type}) : \mathcal{B}\,A \to \mathsf{Type}\ \text{ where} \\
\quad \mathsf{leaf} : \mathsf{T}\ X\ [] \\
\quad \_|\lceil\_\rceil|\_ : \{i_l\ i_r : \mathcal{B}\,A\} \to (u : X\ i_l) \to (p : A) \to \\
\qquad (v : X\ i_r) \to \mathsf{T}\ X\ (p :: i_l \mathbin{+\!\!+} i_r) \\
\text{pattern } \_\hat{}\_|\lceil\_\rceil|\_\hat{}\_\ u\ i_l\ p\ v\ i_r = \_|\lceil\_\rceil|\_\ \{i_l\}\ \{i_r\}\ u\ p\ v
\end{array}
$$

$$
\mathsf{T}\text{-}\varepsilon^{-1} : \{X : \mathcal{B}\,A \to \mathsf{Type}\} \to (i : \mathcal{B}\,A) \to \mathsf{T}\ X\ i \to \mathsf{T}\ (\mathsf{J}\!<\ i\ (X\ |\!<\ i))\ i
$$

(2) Inversion **(1) Pattern match**

$$
\begin{array}{l}
\mathsf{T}\text{-}\varepsilon^{-1}\ .[]\ \ \mathsf{leaf} = \mathsf{leaf} \\
\mathsf{T}\text{-}\varepsilon^{-1}\ .(p :: i_l \mathbin{+\!\!+} i_r)\ (u\,\hat{}\,i_l\,|\lceil\,p\,\rceil|\,v\,\hat{}\,i_r) = \\
\quad ((\mathsf{i}_l\!<\!\mathsf{i}\ ,\ u)\,|\lceil\,p\,\rceil|\,(\mathsf{i}_r\!<\!\mathsf{i}\ ,\ v))\ \text{where } \mathsf{i}_l\!<\!\mathsf{i} : (i_l <_{\#} p :: i_l \mathbin{+\!\!+} i_r)\ ;\ \mathsf{i}_r\!<\!\mathsf{i} : (i_r <_{\#} p :: i_l \mathbin{+\!\!+} i_r)
\end{array}
$$

1. Pattern match on the value of type $\mathsf{T}\ X\ i$;
2. by *inversion* (Dybjer '94), this will refine the original index (seen here as *dot patterns*);

Going Back to Definition-Time with Inversion

$$\begin{array}{l}
\text{data } \mathsf{T}\ (X : \mathcal{B}\,A \to \mathsf{Type}) : \mathcal{B}\,A \to \mathsf{Type} \text{ where} \\
\quad \mathsf{leaf} : \mathsf{T}\ X\ [] \\
\quad _|\lceil_\rceil|_ : \{i_l\ i_r : \mathcal{B}\,A\} \to (u : X\ i_l) \to (p : A) \to \\
\qquad (v : X\ i_r) \to \mathsf{T}\ X\ (p :: i_l +\!\!+ i_r) \\
\text{pattern } _\hat{}_|\lceil_\rceil|_\hat{}_\ u\ i_l\ p\ v\ i_r = _|\lceil_\rceil|_\ \{i_l\}\ \{i_r\}\ u\ p\ v
\end{array}$$

$$\mathsf{T}\text{-}\varepsilon^{-1} : \{X : \mathcal{B}\,A \to \mathsf{Type}\} \to (i : \mathcal{B}\,A) \to \mathsf{T}\ X\ i \to \mathsf{T}\ (\mathsf{J}\!<\ i\ (X\ |\!<\ i))\ i$$

(2) Inversion (1) Pattern match (3) Subproofs that bags i_l, i_r are smaller

$$\begin{array}{l}
\mathsf{T}\text{-}\varepsilon^{-1}\ .[]\quad \mathsf{leaf} = \mathsf{leaf} \\
\mathsf{T}\text{-}\varepsilon^{-1}\ .(p :: i_l +\!\!+ i_r)\ (u\,\hat{}\,i_l\,|\lceil p \rceil|\,v\,\hat{}\,i_r) = \\
\quad ((i_l\!<\!i\ , u)\,|\lceil p \rceil|\,(i_r\!<\!i\ , v)) \text{ where } i_l\!<\!i : (i_l <_{\#} p :: i_l +\!\!+ i_r)\ ;\ i_r\!<\!i : (i_r <_{\#} p :: i_l +\!\!+ i_r)
\end{array}$$

1. Pattern match on the value of type $\mathsf{T}\ X\ i$;
2. by *inversion* (Dybjer '94), this will refine the original index (seen here as *dot patterns*);
3. prove that the indices in the functorial positions are smaller than the original, now refined, outer index.

Structure

Inclusion-restriction adjunction

For our original $(J_{<i} X)_j : \mathcal{C}^{<i} \to \mathcal{C}^W$ we have:

$$\begin{tikzcd} \mathcal{C}^{<i} \arrow[r, bend left, "J_{<i}"] \arrow[r, phantom, "\perp"] & \mathcal{C}^W \arrow[l, bend left, "-|_{<i}"] \end{tikzcd}$$

However, for J<, only the counit can be defined. It turns out that our formalization dualizes to a *partial element* formulation.

```
J< J'< : (i : A) → ((< i) → Type) → (A → Type)
J'< i X j = ∀ (pf : j < i) → X (j , pf)
J<  i X j = Σ[ pf ∈ j < i ] X (j , pf)
```

Dually for J'< only the unit can be defined. Intuition: partial element defer checking totality to *elimination*; copartial elements must be shown total at introduction.

Future Work

- Explore the (co)partial element connection further to un-specialize from Set
- Develop a variant of our technique for an entirely non-indexed (extrinsic) setting

Structure

Conclusion

- We have a new way to express a broad class of algorithms describable as coalgebra-to-algebra morphisms in a total functional programming language. More specifically, a novel sufficient criterion for proving & formalizing recursivity of coalgebras.

Conclusion

- We have a new way to express a broad class of algorithms describable as coalgebra-to-algebra morphisms in a total functional programming language. More specifically, a novel sufficient criterion for proving & formalizing recursivity of coalgebras.
- Notable use case: Indices already used for proving functional properties intrinsically can also serve as a termination measure.

Conclusion

- We have a new way to express a broad class of algorithms describable as coalgebra-to-algebra morphisms in a total functional programming language. More specifically, a novel sufficient criterion for proving & formalizing recursivity of coalgebras.
- Notable use case: Indices already used for proving functional properties intrinsically can also serve as a termination measure.
- General equational definitions, with the possibility to use facilities for generic programming for the remaining boilerplate

$$a\ i \circ F_1\ (\mathrm{iuncurry}\ i\ IH)\ i \circ Fwf\ \ i \circ c\ i$$

Conclusion

- We have a new way to express a broad class of algorithms describable as coalgebra-to-algebra morphisms in a total functional programming language. More specifically, a novel sufficient criterion for proving & formalizing recursivity of coalgebras.
- Notable use case: Indices already used for proving functional properties intrinsically can also serve as a termination measure.
- General equational definitions, with the possibility to use facilities for generic programming for the remaining boilerplate

$$a\ i \circ F_1\ (\mathrm{iuncurry}\ i\ IH)\ i \circ Fwf\ \ i \circ c\ i$$

- More applications & corollaries in our PLDI paper (formalized: correct GCD, CYK)

Contact

Our PLDI '26 paper:

- Website: `www.cxandru.ee` (these slides are already there!)
- Mastodon: `@cxandru@types.pl`
- Mail: `c.alexandru@cs.rptu.de`
- Discord: `@cxandru`